MÉMO

Machine Learning

Ce que je retiens — de Python à la descente de gradient

Réflexes transférables pour la suite du cours et le métier d'ingénieure IA

Quatre axes à garder en tête

Le fil rouge de tout ce qui a été vu

1 Le coût

Combien d'opérations ? Un code juste peut être inutilisable à grande échelle.

2 La forme

shape avant tout. La majorité des bugs viennent de dimensions qui ne correspondent pas.

3 Le silence

Les erreurs les plus coûteuses ne plantent pas : elles renvoient un chiffre faux.

4 La question

Le choix de la métrique décide de la conclusion. Toujours savoir ce qu'on mesure.

Penser au coût, pas seulement au résultat

La complexité dit comment le temps de calcul grandit avec la taille des données

Complexité	Comportement	Exemple rencontré
O(1)	Constant	Accès à une clé de dictionnaire
O(log n)	Divisé par 2 à chaque étape	Recherche dichotomique
O(n)	Proportionnel	Parcours d'une liste
O(n log n)	Optimal pour un tri	Tri fusion
$O(n^2)$	Explose vite	Deux boucles imbriquées

1 million d'éléments

$O(n^2)$

plusieurs heures

O(n log n)

une fraction de seconde

Les deux réflexes qui reviennent partout

Sortir dès qu'on sait (early exit) — inutile de continuer quand la réponse est acquise.
Exploiter ce qu'on sait déjà — liste triée, données déjà dédoublonnées, moitiés déjà triées.

Vectoriser plutôt que boucler

Le prolongement direct du réflexe de coût, dans le monde NumPy

Python pur

```
resultat = []
for x in liste:
    resultat.append(x * 2)
```

La boucle tourne dans l'interpréteur

NumPy

```
resultat = tableau * 2
```

Le calcul descend dans du C compilé

× 10

plus rapide (mesuré, 5 M d'éléments)

÷ 10

de mémoire : octets bruts, pas des objets Python

0

boucle visible dans le code

NumPy — les quatre réflexes

Le socle de tout le calcul en machine learning

1

shape, toujours

.shape est l'attribut le plus consulté. La plupart des erreurs viennent de formes qui ne correspondent pas.

```
M.shape  →  (3, 4)
```

2

Broadcasting

NumPy aligne les formes en partant de la droite. Compatibles si égales, ou si l'une vaut 1. Un (3,) n'a pas d'orientation.

```
(3,4) + (4,)   ✓
(3,4) + (3,)   ✗   → v[:, None]
(3,4) + (3,1)  ✓
```

3

axis = l'axe qui disparaît

Une agrégation réduit d'une dimension. keepdims=True conserve l'axe à la taille 1 pour rebroadcaster.

```
C.mean(axis=0)  →  (4,)
C.mean(axis=1, keepdims=True)
```

4

Masques booléens

Une comparaison renvoie un tableau. On filtre avec &, | et ~ — jamais and/or — et parenthèses obligatoires.

```
a[(a > 3) & (a < 9)]
```

Pandas — les gestes qui reviennent

Découvrir un jeu de données, le nettoyer, l'interroger

Découvrir

```
df.shape
df.head()
df.info()
df.describe()
df.isna().sum()
```

info() dit ce qu'on a, describe() où sont les valeurs aberrantes.

Sélectionner

```
df["col"]        → Series
df[["a","b"]]     → DataFrame
df.loc[2, "nom"]  étiquette
df.iloc[0:3]      position
```

Le double crochet est une liste : le résultat reste en 2D.

Interroger

```
df[df["age"] > 30]
df.groupby("ville")["x"].mean()
df.groupby("x").agg(["mean","count"])
```

Toujours count à côté de mean : une moyenne sans effectif ne veut rien dire.

Les pièges silencieux des données réelles

Aucun ne provoque d'erreur — tous produisent un résultat faux

Espaces invisibles

Les exports à format fixe laissent des espaces de fin. "GO " ≠ "GO", et l'affichage ne montre rien.

```
.str.strip() dès le chargement
```

Filtre qui ne renvoie rien

Un tableau vide se propage : .mean() donne nan, .max() lève une erreur trois étapes plus loin.

```
tester .size > 0
```

Moyenne sans effectif

Une moyenne sur 1 élément s'affiche comme les autres. Rien ne signale qu'elle ne veut rien dire.

```
agg(["mean", "count"])
```

nan ignoré par défaut

mean() saute les valeurs manquantes : le dénominateur change sans avertissement.

```
isna().sum() avant tout calcul
```

Valeurs manquantes : pourquoi est-elle absente ?

La bonne question n'est pas « comment je la comble ? »

Absence structurelle

L'absence porte une information : la mesure n'existe pas pour cette catégorie.

- Deux colonnes complémentaires selon la norme appliquée
- Une grandeur nulle par nature, laissée vide

Un dropna() global supprimerait une catégorie entière.

Absence accidentelle

La donnée existe mais n'a pas été renseignée.

- Quelques pourcents épars, sans motif
- Aucun lien avec une catégorie

Là, dropna() ou fillna() se discutent.

Croiser les valeurs manquantes avec une variable catégorielle révèle le motif — et évite de biaiser toute l'analyse.

Le triptyque de l'apprentissage

Trois briques distinctes — le même mécanisme entraîne un réseau de neurones

1

Le modèle

$f(x) = wx + b$

Prend un x, suppose w et b connus, produit une prédiction.

→

2

La fonction de coût

$J(w,b)$

Mesure à quel point ce couple (w, b) est mauvais. Moyenne des erreurs au carré.

→

3

La descente de gradient

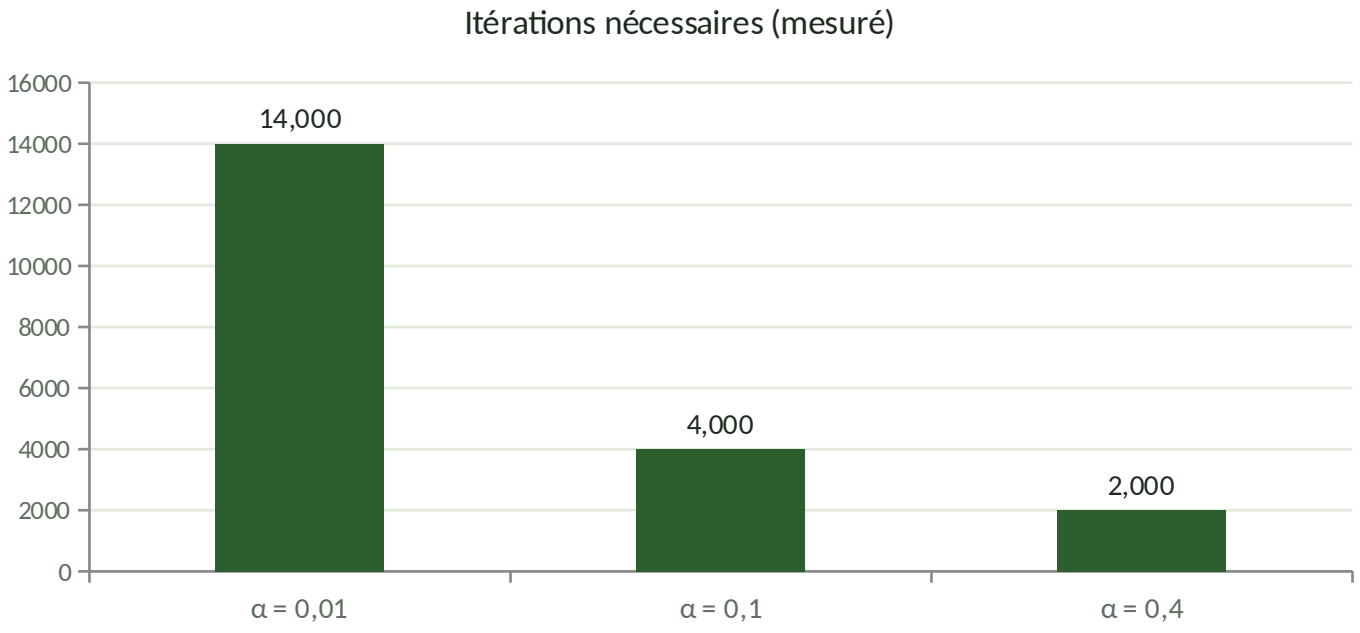
$w \leftarrow w - \alpha \, \partial J/\partial w$

Ajuste w et b pas à pas dans le sens de la pente, jusqu'au minimum.

En pratique : LinearRegression().fit(X, y) condense les trois. Comprendre le mécanisme reste indispensable — il se généralise.

Le taux d'apprentissage vit dans une fenêtre étroite

Trop petit : on n'arrive jamais. Trop grand : tout diverge en nan.



α = 0,001

20 000 itérations et toujours pas arrivé. Ça avance, mais dix fois trop lentement.

α = 0,46

Le coût explose au lieu de baisser : inf, puis nan. Un nan contamine tout ce qui suit.

Tracer le coût en fonction des itérations est le diagnostic de base : il doit décroître à chaque pas.

La normalisation ne sert pas qu'au multi-variables

Centrer x découple w et b — la vallée étroite redevient un bol

Sans normalisation

x n'est pas centré : augmenter w relève toutes les prédictions, b doit compenser. Les deux paramètres sont couplés — le coût forme une vallée longue et étroite.

Avec normalisation

$(x - \mu) / \sigma \rightarrow$ les directions deviennent indépendantes, la descente va tout droit. Et un α bien plus grand devient utilisable sans diverger.

> 20 000
itérations sans normaliser

→

< 200
itérations après normalisation, à α identique

Conserver μ et σ : les paramètres appris portent sur la variable transformée. Il faut les reconvertir pour retrouver des unités interprétables.

Évaluer un modèle : combien, et surtout où

Un score global ne dit jamais où le modèle échoue

R^2

Part de la variation expliquée. En régression simple, c'est exactement le carré de la corrélation.

MAE / RMSE

L'erreur moyenne, dans l'unité de la cible. Directement interprétable, contrairement au coût J.

Les résidus

Le graphique le plus informatif. Des erreurs qui s'évasent ou une traînée asymétrique désignent une variable manquante.

Le calcul du R^2 dépend de la population choisie

Mélanger deux sous-populations brouille la relation ; isoler une catégorie peut la révéler beaucoup plus nette. Un même indicateur calculé sur deux périmètres différents n'est pas comparable.

Corrélation, pente, causalité

Trois notions souvent confondues

1

La corrélation

Mesure l'alignement des points. Sans unité, toujours entre −1 et 1, insensible au changement d'échelle.

2

La pente

Mesure de combien y varie quand x augmente d'une unité. A une unité, dépend des échelles choisies.

3

Ni l'une ni l'autre n'est une cause

Une corrélation forte peut refléter une identité physique, une variable cachée, ou une coïncidence.

Une corrélation faible ne signifie pas absence de relation : le coefficient ne mesure que la linéarité. Toujours regarder le nuage de points.

Une variable trop bonne pour être vraie

Un R^2 de 0,999 n'est pas une bonne nouvelle

Fuite directe

Le CO_2 homologué est calculé à partir de la consommation : leur rapport est une constante.

- R^2 = 0,999 et MAE = 0,9 g/km
- Coefficient appris : 2,64 kg de CO_2 par litre

Le facteur d'émission du gazole — une constante physique, pas un paramètre.

Fuite par formule

La puissance administrative se calcule depuis 1998 par une formule qui contient le CO_2.

- Corrélation avec la cible : 0,708 seulement
- La formule reproduit 99,5 % des lignes

Une fuite non linéaire est invisible dans une matrice de corrélation.

Le test : cette variable serait-elle connue avant la cible, en situation réelle ? Sinon, elle sort du modèle.

La préparation fait partie du modèle

Une fuite peut venir du code, pas seulement des données

Le symptôme

Un score de test trop beau, jamais retrouvé ensuite. Rien ne plante, rien n'avertit : le chiffre est simplement faux.

La cause

μ et σ calculés sur tout X avant le découpage. L'information du jeu de test entre dans la préparation, donc dans le modèle.

Le remède

Le pipeline : la préparation et l'estimateur deviennent un seul objet, qui s'ajuste d'un bloc sur le seul jeu d'entraînement.

Séquentiel ou parallèle

Pipeline enchaîne : la sortie d'une étape est l'entrée de la suivante. ColumnTransformer découpe : chaque branche ses colonnes, sorties concaténées. Une liste de paires (nom, objet) des deux côtés — ces noms préfixent les hyperparamètres.

Une ligne n'est pas une observation

Vérifier l'unité du fichier avant tout calcul

Ce que contient le fichier

Chaque véhicule est déclaré une fois par variante d'homologation : mêmes masse, puissance et CO_2, numéros de dossier différents. Rien ne signale la répétition.

Ce que ça fait au modèle

Un modèle déclaré en 18 versions pèse 18 fois plus lourd dans la somme des carrés. L'ajustement suit le catalogue commercial, pas la physique.

6 934
lignes essence dans le fichier

→

2 406
véhicules après dédoublonnage

Dédoublonner sur des clés métier explicites — et dire lesquelles. Ici : marque, modèle, masse, puissance, CO_2.

Quand deux variables se recouvrent

La colinéarité ne fausse pas les prédictions — elle interdit de lire les coefficients

Le symptôme

Un coefficient au signe physiquement absurde : à masse égale, plus le véhicule serait puissant, moins il émettrait.

Le diagnostic

Dans l'ordre : corrélation entre variables, R^2 de chacune seule, puis gain incrémental. Un gain nul = le coefficient ajuste du bruit.

La conséquence

Les coefficients ne s'interprètent plus un par un. Seule leur combinaison prédit — et le modèle reste utilisable.

Les R^2 ne s'additionnent pas

0,6158 pour la masse seule, 0,7618 pour la puissance seule, 0,7886 pour les deux ensemble. L'information commune est partagée, pas additive : c'est le gain incrémental qui compte, jamais le R^2 d'une variable prise seule.

Le R^2 d'entraînement ne peut que monter

Dès que le nombre de paramètres varie, il cesse d'être un critère de comparaison

Le mécanisme

Une variable de plus donne un degré de liberté de plus. Même du bruit pur fait monter le R^2 d'entraînement.

Ce que ça masque

Rien ne distingue alors l'information réelle de l'ajustement au bruit. Le ratio inutile et la variable décisive montent tous les deux.

Ce qui le remplace

Évaluer sur des données non vues : train_test_split puis R^2 de test. Ou un R^2 ajusté, qui pénalise le nombre de paramètres.

Un R^2 de test supérieur au R^2 d'entraînement n'est pas une bonne nouvelle

Il signale un tirage favorable, pas un bon modèle. Le classement de plusieurs modèles reste valide s'ils partagent le même découpage ; les valeurs absolues, non. La validation croisée tranche.

Comparer à quoi ?

Une erreur ne se juge que relativement

La référence naïve

Prédire la moyenne du parc pour tous les véhicules, sans regarder aucune variable. C'est le modèle le plus bête possible — et le point de départ obligé.

Ce qu'elle révèle

15,5 g/km d'erreur ne veut rien dire seul. Rapporté à la référence, ça devient un résultat : masse, puissance et boîte divisent l'erreur par 2,8.

42,9 g/km
erreur de la référence naïve

→

15,5 g/km
erreur du modèle, sur le jeu de test

Même réflexe en classification : la classe majoritaire. Un modèle qui ne bat pas la référence naïve n'a rien appris.

Les résidus, deuxième lecture

Ce que le nuage dit quand on sait quoi y chercher

1 La dispersion s'évase

L'erreur passe de 11,5 g/km en bas de gamme à 31,7 au-dessus de 300 kW. L'incertitude dépend du véhicule — le R^2 ne le dit pas.

2 La traînée est d'un seul côté

Les vingt plus gros résidus sont tous positifs. Le modèle sous-estime sans jamais surestimer autant : signature d'une variable manquante.

3 Une courbure peut rester

Moyenne des résidus par décile de prédiction. Un motif en U = non-linéarité non captée. L'œil seul surinterprète les nuages denses.

4 Attention aux zéros mécaniques

Le résidu moyen par modalité d'une variable présente dans le modèle est nul par construction. Ça ne démontre rien.

Hygiène de travail

Ce qui distingue un brouillon d'un livrable

Environnement

- Un dossier = un projet = un .venv
- Ouvrir le projet, pas le dossier parent
- which python quand un import échoue
- requirements.txt : dépendances directes

Notebook

- Restart and Run All avant chaque commit
- Une variable réutilisée = état fantôme
- Numéros d'exécution dans l'ordre
- Chaque chiffre affiché doit être mesuré

Git & portfolio

- add / commit / push, à chaque étape finie
- .venv ignoré, jamais versionné
- README : question, méthode, résultat, limites
- Nommer les limites de son analyse

Classer, ou prédire puis seuiller

Deux chemins vers la même décision binaire — et ils se valent

Classer directement

Le modèle ajuste la frontière elle-même et reste conservateur. Il est plus précis : quand il annonce un malus, il se trompe moins souvent.

Precision 88,6 % · Recall 85,4 %

Prédire, puis seuiller

Le modèle prédit des grammes. Son erreur moyenne (≈ 13 g/km) lui fait franchir le seuil plus facilement : il manque moins de malus réels.

Precision 85,3 % · Recall 88,8 %

87,0 %
F1 de la classification directe

=

87,0 %
F1 de la régression + seuil

Vérifié sur trois découpages : l'écart entre les deux approches est plus petit que la variation d'un tirage à l'autre. Ce qui les sépare n'est pas la quantité d'erreurs, mais leur nature — donc le choix relève du coût métier.

Precision, recall, F1

Trois questions différentes — tout se joue sur l'ensemble d'où l'on part

Les quatre cas

	prédit positif	prédit négatif
réel positif	VP	FN
réel négatif	FP	VN

FP = fausse alerte · FN = cas manqué

```
Precision = VP / (VP + FP)
```

part du PRÉDIT positif — quand j'annonce, ai-je raison ?

```
Recall    = VP / (VP + FN)
```

part du RÉEL positif — combien en ai-je trouvés ?

```
F1        = 2PR / (P + R)
```

moyenne harmonique — élevée seulement si les deux le sont

72,6 %

F1 du naïf, qui prédit « malus » partout

→

87,0 %

F1 du modèle : +14 points, et non +30

F1 ignore complètement les vrais négatifs : elle ne regarde qu'autour de la classe positive. L'accuracy les compte — les deux sont complémentaires, pas redondantes.

L'erreur n'est pas uniforme

Taux d'erreur des deux approches, par distance au seuil du malus

≈ 41 %	≈ 36 %	≈ 13 %	≈ 3 %
moins de 5 g	**5 à 10 g**	**10 à 20 g**	**plus de 20 g**
163 véhicules	129 véhicules	249 véhicules	556 véhicules

Chercher le régime, pas la moyenne

Quand une décision binaire dérive d'une grandeur continue, les erreurs se concentrent près du seuil : un facteur 12 à 15 entre les deux extrêmes. Un score global de 85 % moyennait deux régimes — fiable loin de la frontière, quasi aléatoire à moins de 5 g.

- Une PRÉDICTION près du seuil n'est pas une valeur réelle près du seuil : un véhicule à 216 g estimé à 132 g tombe du bon côté par chance.

Le fil rouge

Un résultat juste ne suffit pas

- Il doit être calculable à l'échelle réelle — d'où le coût et la vectorisation.
- Il doit être vérifié, car l'erreur la plus dangereuse ne plante pas.
- Il doit être situé : sur quelle population, avec quelle métrique, dans quelles limites.
- Il doit être reproductible par quelqu'un d'autre.

Même en classification, le fil rouge ne change pas : vérifier, situer, rendre reproductible.